



Création et utilisation de la base de données

« Base immobilière » du projet
DATAIMMO



Laplace Immo

Contexte du projet

Le directeur général de **Chez Laplace Immo** (réseau national d'agences immobilières) souhaite que l'agence se démarque de la concurrence en créant un modèle pour mieux prévoir **le prix de vente des biens immobiliers**.

Ce projet stratégique est appelé en interne le projet "**DATAIMMO**".

La **CTO** (Chief Technology Officer) de l'entreprise nous a confié la mise en oeuvre du **POC** (Proof of Concept). Il s'agit de créer **un schéma de base de données relationnelle** à partir d'un **nouveau dictionnaire de données de référence** afin de répondre précisément à **12 questions du métier** sur les données de **vente du premier semestre 2020**.

Ce **POC** s'il est validé par le **CEO** (Chief executive Officer) permettra de **généraliser** plus tard aux **données des années précédentes**. Ce qui servira peut-être cette base de **données d'entraînement** au modèle de **prédiction des prix de vente**.

La stratégie de sauvegarde et la conformité RGPD

Conformité RGPD :

- Les données provenant de sources **publiques** (**Data gouv**, **BDPF**, **Dgfip**) **ne sont pas exhaustives et ont fait l'objet d'un filtrage pour les besoins.**
- Néanmoins, le fichier **DVF** contient des données à caractère personnel qui renseigne directement et indirectement sur la vie privée des individus puisqu'il permet l'association d'une adresse postale à un nom patronymique associée à des données géographiques de region et population.
- ☛ C'est la raison pour laquelle , nous ne garderons pas la donnée « **Nom de l'acquéreur** » dans notre dictionnaire de données de reference.



Sauvegarde :

- Les **données extraites** des 3 fichiers .csv ainsi que le **Schéma** intitulé « **base immobilière** » sont stockées sur le serveur MySQL par **mot de passe sécurisé**. et accessibles depuis un client Dbeaver .Les données sont sauvegardées en **Local** et en **France**.

Les données initiales

DICTIONNAIRE DES DONNÉES - Valeurs foncières

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL
Id_bien	ID dans la base de données	Integer	NC	Calculé	Unique, Ne doit pas être nul	AUTO-INCREMENT
No_voie	Numéro des rues	Integer	NC	Elémentaire	Ne doit pas être nul	
Type_de_voie	Plusieurs valeurs (rue, avenue, chemin, etc.)	Varchar	4	Elémentaire	Ne doit pas être nul	
Voie	Nom de la rue	Varchar	50	Elémentaire	Ne doit pas être nul	
BTQ	Code Batiment, Terrain, Quartier	Varchar	1	Elémentaire	Ne doit pas être nul	
Total_piece	Nombre de pièces principales à vivre	Integer	3	Elémentaire	Ne doit pas être nul	
Surface_carrez	Nombre de mètres carrés en Surface	Decimal	8	Elémentaire	Ne doit pas être nul	
Surface_local	Nombre de mètres carrés réels du bati	Integer	3	Elémentaire	Ne doit pas être nul	
Type_local	Libellé Appartement ou Maison	Varchar	50	Elémentaire	Ne doit pas être nul	
Id_vente	ID dans la base de données	Integer	NC	Calculé	Unique, Ne doit pas être nul	AUTO-INCREMENT
Date	Date de la nature de la mutation (Vente)	Date	10	Elémentaire	Au format Date (AAAA/MM/JJ)	
Valeur	Valeur marchande en euros du foncier	Decimal	8	Elémentaire	Ne doit pas être nul	
Code_postal	Code postal	Integer	5	Elémentaire	Ne doit pas être nul	
Nom_commune	nom de la Commune	Varchar	50	Elémentaire	Ne doit pas être nul	
Code_departement	Code du département	Integer	3	Elémentaire	Ne doit pas être nul	
Code_commune	Code de la Commune	Integer	3	Elémentaire	Ne doit pas être nul	

L'extrait du dictionnaire des données modifiées

DICTIONNAIRE DES DONNÉES - Valeurs foncières

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE D
Id_bien	ID dans la base de données	Integer	NC	Calculé	PRIMARY Key, NOT NULL	AUTO-IN
BTQ	Code Batiment, Terrain, Quartier	Varchar	1	Elémentaire		
Total_pieces	Nombre de pièces principales composant l'habitation	Integer		Elémentaire		
No_voie	Numéro des rues	Integer	4	Elémentaire		
Type_voie	Plusieurs valeurs (rue, avenue, chemin, etc.)	Varchar	20	Elémentaire		
Voie	Libellé de la voie	Varchar	50	Elémentaire		
Surface_local	Nombre de mètres carrés réels du bati en mètres carrés associée au local	Decimal	3	Elémentaire		
Lot1	identification du Lot (EDD) permet d'identifier une partie d'un immeuble et d'y associer un droit de propriété spécifique (propriété, usufruit etc...)	Varchar	4	Elémentaire		
Surface_carrez	Total nombre de mètres carrés en surface (associée au lot dans la limite de 5)	Decimal	8	Elémentaire		
No_Plan	Numéro du bien sur le Plan cadastral	Integer	4	Elémentaire		
Section	Numéro de section cadastrale	Varchar	2	Elémentaire		
Prefixe_de_Section	Préfixe de numéro de section cadastrale	Varchar	3	Elémentaire		
biencodetypelocal	Code du type de local dont 1 ou 2	Integer	1	Elémentaire	FOREIGN Key, NOT NULL	
Depcode	Code du département	Integer	3	Elémentaire	FOREIGN Key, NOT NULL	
comcode	Code INSEE de la Commune	Varchar	5	Composite	FOREIGN Key, NOT NULL	
Identifiant_vente	ID dans la base de données	Integer	NC	Calculé	PRIMARY Key, NOT NULL	AUTO-IN
Nature	Nature de la mutation immobilière à titre onéreux des 5 dernières années (vente, échange, expropriation ou adjudication) ici juste Vente	Varchar		Elémentaire		
Date	Date de signature de l'acte (au format JJ/MM/AAAA) – Une restitution au format AAAA/MM/JJ (norme ISO 8601) est prévue à compter de la mise à jour d'octobre 2019. (PI, sur 10 semestres en avril, sur 11 semestres en octobre)	Varchar	10	Elémentaire		

Le schéma relationnel normalisé

P5_Schema_relationnel.architect2 - SQL Power Architect

File Edit Connections ETL OLAP Enterprise Tools Profile Window Help



New Project

P5_Schema_relationnel.architect2

Database

(TABLE)

mune (TABLE)

ie (TABLE)

ilation (TABLE)

mns folder for Population

Identifiant_population: INTEGER [PK]

com_code: VARCHAR(3) [FK]

com_coddep: VARCHAR(3) [FK]

total_hab: INTEGER

popu_active: INTEGER

popu_totale: INTEGER

rted keys folder for Population

rted keys folder for Population

es folder for Population

on (TABLE)

z_local (TABLE)

artement (TABLE)

Type_local

codetypelocal: INTEGER NOT NULL [PK]

type_local: VARCHAR(20) NOT NULL

Bien

Id_bien: INTEGER NOT NULL [PK]

BTQ: VARCHAR(1) NOT NULL

biencodetypelocal: INTEGER NOT NULL [FK]

comcode: VARCHAR(3) NOT NULL [FK]

Dep_code: VARCHAR(3) NOT NULL [FK]

Lot1: VARCHAR NOT NULL

No_Plan: INTEGER NOT NULL

No_voie: VARCHAR(4) NOT NULL

Prefixe_de_Section: VARCHAR(6) NOT NULL

Section: CHAR(2) NOT NULL

Surface_carrez_lot: DECIMAL(8) NOT NULL

Surface_local: DECIMAL(8) NOT NULL

Total_pieces: INTEGER NOT NULL

Type_voie: VARCHAR(20) NOT NULL

voie: VARCHAR(50) NOT NULL

Vente

Id_vente: INTEGER NOT NULL [PK]

Nature_mutation: VARCHAR(30) NOT NULL

Date_mutation: DATE NOT NULL

Valeur_fonciere: DECIMAL NOT NULL

Modisposition: INTEGER NOT NULL

Identifiant_bien: INTEGER NOT NULL [FK]

Departement

dep_code: VARCHAR(3) NOT NULL [PK]

dep_reg_cod: VARCHAR(2) NOT NULL [FK]

dep_nom: VARCHAR(50) NOT NULL

Region

reg_code: VARCHAR(2) NOT NULL [PK]

reg_nom: VARCHAR(50) NOT NULL

reggrp_nom: VARCHAR(30) NOT NULL

Commune

com_coddep: VARCHAR(3) NOT NULL [PK]

com_code: VARCHAR(3) NOT NULL [FK]

id_codedep_codcom: VARCHAR(5) NOT NULL

com_nom: VARCHAR(50) NOT NULL

Population

Identifiant_population: INTEGER NOT NULL [PK]

com_code: VARCHAR(3) NOT NULL [FK]

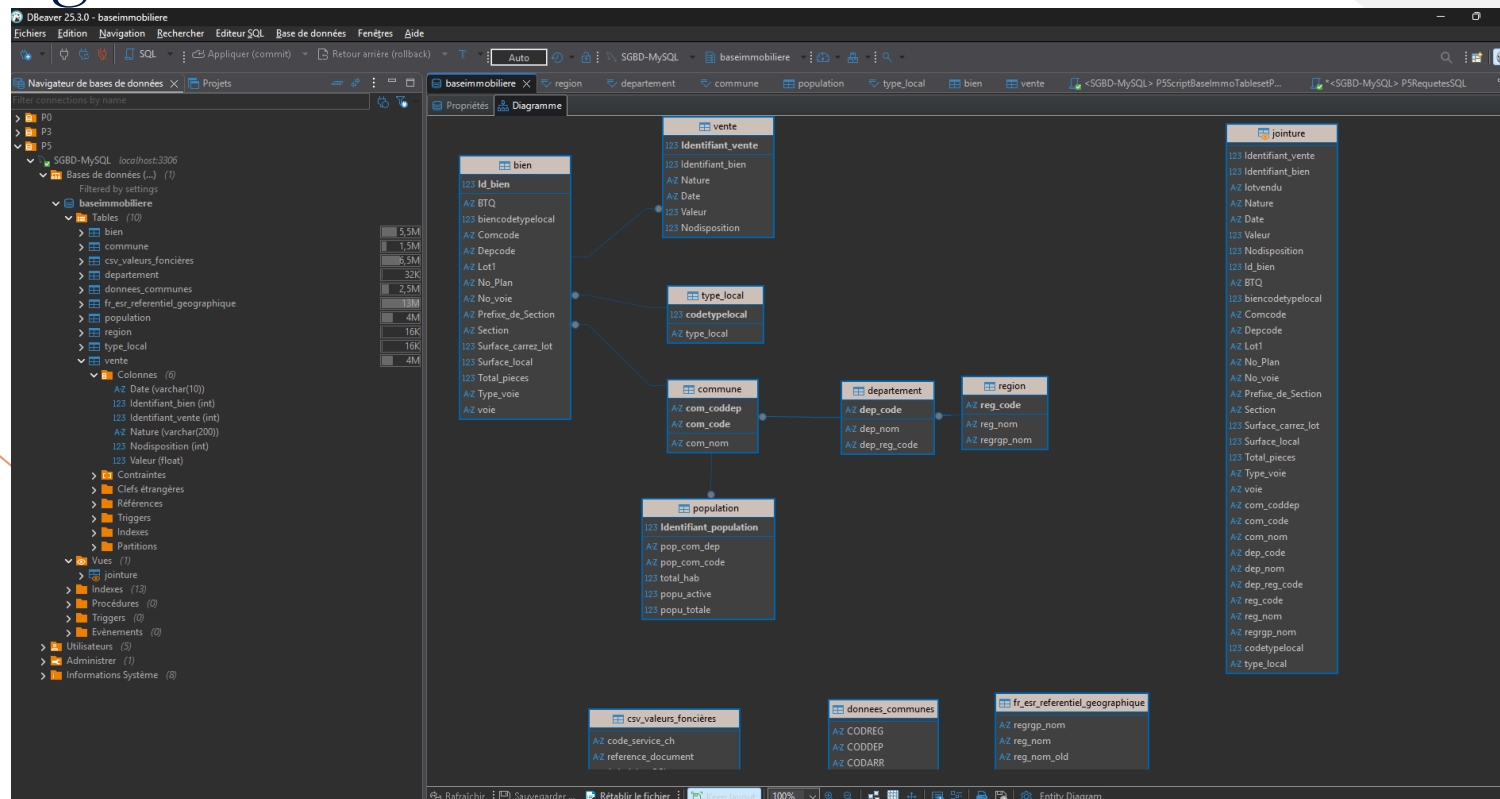
com_coddep: VARCHAR(3) NOT NULL [FK]

total_hab: INTEGER NOT NULL

popu_active: INTEGER NOT NULL

popu_totale: INTEGER NOT NULL

La base de données avec les tables créées et les données chargées



Les requêtes ou screenshot qui permettent de démontrer le bon chargement des données

The screenshot displays the DBeaver 25.3.0 interface. On the left, the 'Navigateur de bases de données' (Database Navigator) shows a tree structure of the 'SGBD-MySQL' database. The 'Tables' folder is highlighted with a yellow circle. The 'baseimmobiliere' table is selected, and its properties are shown on the right. The main window displays the 'baseimmobiliere' table view, showing columns: 'id_bien', 'AZ BTQ', 'biencodetypelocal', 'AZ Comcode', 'AZ Depcode', 'AZ Lot1', 'AZ No_Plan', 'AZ No_voie', 'AZ Prefixe_de_Section', and 'AZ Section'. The status bar at the bottom indicates '34169 row(s) fetched - 0,107s (0,049s fetch), on 2026-01-20 at 21:32:35'.



Requêtes SQL et résultats

Base immobilière MySQL du projet DataImmo

Requête 1

« Nombre total d'appartements vendus au 1er semestre 2020 »

The screenshot shows the DBeaver 25.3.0 interface. The left sidebar displays the database structure for 'SGBD-MySQL' on 'localhost:3306'. The 'Tables' folder is expanded, showing tables like 'bien', 'commune', 'csv_valeurs_foncieres', 'departement', 'donnees_communes', 'fr_esr_referentiel_geographique', 'population', 'region', 'type_local', and 'vente'. The 'Columns' panel for the 'vente' table is visible, listing columns: 'A2 Date (varchar(10))', '123 Identifiant_bien (int)', '123 Identifiant_vente (int)', 'A2 Nature (varchar(200))', '123 Nodisposition (int)', and '123 Valeur (float)'. The main editor shows a SQL query:

```
/*1. Le Nombre total d'appartements vendus au 1er semestre 2020*/
select distinct tl.type_local, count(*) as N
from vente v
join bien b on b.Id_bien = v.Identifiant_vente
join type_local tl on b.biencodetypelocal = tl.codetypelocal
where b.biencodetypelocal = 2
and MONTH(v.date) in (1,2,3,4,5,6)
order by N desc
;
```

The query results are displayed in a table with two columns: 'type_local' and 'N'. The first row shows 'Appartement' with a count of 31378.

type_local	N
Appartement	31378

The status bar at the bottom indicates '1 row(s) fetched - 0,044s, on 2026-01-22 at 16:07:03'.

Requête 2

Nombre de ventes d'appartement par région pour le 1er semestre 2020

The screenshot shows the DBeaver SQL editor interface. The left sidebar displays the database structure for 'SGBD-MySQL', including tables like 'bien', 'commune', 'departement', 'population', 'region', 'type_local', and 'vente'. The main editor window contains the following SQL query:

```
/*2. Le nombre de ventes d'appartement par région pour le 1er semestre 2020. */
select r.reg_nom, count(*) as N from vente v, bien b
join departement d on d.dep_code = b.dep_code
join region r on r.reg_code = d.dep_reg_code
where b.id_bien = v.identifiant_vente
and b.biencodetypelocal=2
and month(v.Date ) between 1 and 6
group by r.reg_nom
order by N desc;
```

The query results are displayed in a table with 17 rows, showing the number of sales (N) for each region. The results are as follows:

region 1	Sortie
A4 reg_nom	123 N
1 Ile-de-France	13 995
2 Provence-Alpes-Côte d'Azur	3 649
3 Auvergne-Rhône-Alpes	3 253
4 Nouvelle-Aquitaine	1 932
5 Occitanie	1 640
6 Pays de la Loire	1 357
7 Hauts-de-France	1 254
8 Grand Est	984
9 Bretagne	983
10 Normandie	862
11 Centre-Val de Loire	696
12 Bourgogne-Franche-Comté	376
13 Corse	223
14 Martinique	94
15 La Réunion	44
16 Guyane	34
17 Guadeloupe	2

The status bar at the bottom indicates that 17 rows were fetched, with 0.152s execution time on 2026-01-22 at 16:35:23.

Requête 3

Proportion des ventes d'appartements par le nombre de pièces.

The screenshot shows the DBeaver 25.3.0 interface with a MySQL database named 'SGBD-MySQL'. The left sidebar displays the database structure, including tables like 'vente' and 'vente_bien_FK'. The main editor shows a SQL query that calculates the proportion of sales by the number of rooms. The query is as follows:

```
SELECT
  b.Total_pieces,
  CONCAT(
    ROUND((COUNT(*) / NULLIF((SELECT COUNT(*) FROM vente v2 JOIN bien b2 ON v2.Identifiant_vente = b2.Id_bien WHERE b2.Surface_carrez_lot IS NOT NULL), 0)) * 100, 2),
    '%'
  ) AS ProportionPourcentage
FROM
  vente v
JOIN
  bien b ON b.Id_bien = v.Identifiant_vente
WHERE
  b.biencodetypelocal = 2
GROUP BY
  b.Total_pieces
ORDER BY
  b.Total_pieces ASC;
```

The results are displayed in a table with two columns: 'Total_pieces' and 'ProportionPourcentage'. The data is as follows:

Total_pieces	ProportionPourcentage
0	0.09%
1	19.72%
2	28.63%
3	26.24%
4	13.05%
5	3.26%
6	0.60%
7	0.16%
8	0.05%
9	0.02%
10	0.01%
11	0.00%

The bottom status bar indicates that 12 rows were fetched in 0.106s on 2026-01-22 at 16:45:00.

Requête 4

Liste des 10 départements où le prix du mètre carré est le plus élevé.

The screenshot shows the DBeaver 25.3.0 interface with a SQL query executed against a MySQL database. The query is as follows:

```
/*4. Liste des 10 départements où le prix du mètre carré est le plus élevé. 75,92,94 ==> 344 : where surface >0 , */  
SELECT  
  d.dep_code , d.dep_nom as Departement,  
  round(avg(v.Valeur /b.Surface_carrez_lot),2) AS MoyPrixMetrecarre  
FROM  
  vente v  
  join  
    bien b on v.Identifiant_bien = b.Id_bien  
  JOIN  
    departement d ON d.dep_code = b.Depcode  
GROUP BY  
  d.dep_code , d.dep_nom  
ORDER BY  
  MoyPrixMetrecarre desc  
limit 10  
;
```

The results are displayed in a table with the following data:

AZ < dep_code	A2 Departement	MoyPrixMetrecarre
75	Paris	12045,02
92	Hauts-de-Seine	7219,37
94	Val-de-Marne	5343,28
06	Alpes-Maritimes	4700,52
74	Haute-Savoie	4667,13
93	Seine-Saint-Denis	4399,62
78	Yvelines	4225,57
69	Rhône	4059,71
2A	Corse-du-Sud	4026,97
33	Gironde	3766,47

The interface also shows a sidebar with the database structure, including tables like 'bien', 'commune', 'departement', and 'vente'. The bottom status bar indicates that 10 rows were fetched.

Requête 5

Prix moyen du mètre carré d'une maison en Île-de-France.

The screenshot shows the DBeaver 25.3.0 interface with a MySQL database named 'SGBD-MySQL'. The left sidebar displays the database schema, including tables like 'bien', 'commune', 'departement', 'population', 'region', 'type_local', and 'vente'. The main editor displays the following SQL query:

```
/*5. Prix moyen du mètre carré d'une maison en Île-de-France. valeur/surface carrez ==> 3700*/
SELECT
  r.reggrp_nom,
  round(avg(v.Valeur) / avg(b.Surface_carrez_lot),2) AS MoyPrixMetrecarre
FROM
  vente v
  join
    bien b on v.Identifiant_bien = b.Id_bien
  JOIN
    departement d ON d.dep_code = b.Depcode
  join
    region r on r.reg_code = d.dep_reg_code and r.reggrp_nom = 'Île-de-France'
where
  b.biencodetypelocal =1
  and b.Surface_carrez_lot >0
GROUP BY
  r.reggrp_nom
ORDER BY
  MoyPrixMetrecarre desc
```

The results pane shows a single row for 'Île-de-France' with a value of 3675,79 for 'MoyPrixMetrecarre'.

reggrp_nom	MoyPrixMetrecarre
Île-de-France	3675,79

The bottom status bar indicates that 1 row(s) were fetched, and the data was retrieved on 2026-01-22 at 17:15:30.

Requête 6

Liste des 10 appartements les plus chers avec la région et le nombre de mètres carrés.

The screenshot shows the DBeaver SQL editor interface. The top toolbar includes options like 'Auto', 'SQL', 'Appliquer (commit)', and 'Retour arrière (rollback)'. The left sidebar displays a tree view of the database structure, including 'baseimmobiliere', 'region', 'departement', 'commune', 'population', 'type_local', 'bien', and 'vente'. The main editor area contains the following SQL query:

```
SELECT
  r.reg_nom,
  d.dep_code,
  d.dep_nom as departement,
  v.Valeur as valeur,
  b.Surface_local as TotalMetresCarrés
FROM
  vente v
JOIN
  bien b ON v.Identifiant_bien = b.Id_bien
JOIN
  departement d ON d.dep_code = b.Depcode
JOIN
  region r ON r.reg_code = d.dep_reg_code
WHERE
  b.biencodetypelocal = 2 and b.Surface_carrez_lot IS NOT NULL
group by
  r.reg_nom,
  d.dep_code,
  d.dep_nom,
  v.Valeur,
  b.Surface_local
having (select max(v2.valeur) from vente v2)
order by v.valeur desc
limit 10
```

The bottom panel shows the results of the query in a table view. The table has 6 columns: 'A2 reg_nom', 'A2 dep_code', 'A2 departement', '123 valeur', and '123 TotalMetresCarrés'. The results are sorted by '123 valeur' in descending order, showing the top 10 most expensive apartments.

	A2 reg_nom	A2 dep_code	A2 departement	123 valeur	123 TotalMetresCarrés
1	Ile-de-France	75	Paris	9 000 000	10
2	Ile-de-France	91	Essonne	8 600 000	62
3	Ile-de-France	75	Paris	8 577 710	289
4	Ile-de-France	75	Paris	7 620 000	42
5	Ile-de-France	75	Paris	7 600 000	200
6	Ile-de-France	75	Paris	7 535 000	143
7	Ile-de-France	75	Paris	7 420 000	357
8	Ile-de-France	75	Paris	7 200 000	241
9	Ile-de-France	75	Paris	7 050 000	310
10	Ile-de-France	75	Paris	6 600 000	76

The bottom status bar indicates '10 row(s) fetched - 0,22s, on 2026-01-22 at 17:19:27'.

Requête 7

Taux d'évolution du nombre de ventes entre le premier et le second trimestre de 2020.

The screenshot shows the DBeaver 25.3.0 interface with a MySQL database named 'SGBD-MySQL'. The left sidebar displays the database structure, including tables like 'bien', 'commune', 'population', and 'vente'. The main editor contains a SQL query to calculate the percentage change in sales between the first and second quarters of 2020. The query uses a CTE 'VentesParTrimestre' to aggregate sales by quarter and then calculates the percentage change using a formula.

```
/*7. Taux d'évolution du nombre de ventes entre le premier et le second trimestre de 2020.=>3.68 (voir requete with)*/
WITH VentesParTrimestre AS (
  SELECT
    CASE
      WHEN MONTH(v.date) IN (1, 2, 3) THEN 'T1'
      WHEN MONTH(v.date) IN (4, 5, 6) THEN 'T2'
    END AS trimestre,
    COUNT(*) AS nombre_ventes
  FROM
    vente v
  WHERE
    YEAR(v.date) = 2020 and MONTH(v.date) IN (1,2,3,4,5,6)
  GROUP BY
    trimestre
)
SELECT
  MAX(CASE WHEN trimestre = 'T1' THEN nombre_ventes END) AS ventes_t1,
  MAX(CASE WHEN trimestre = 'T2' THEN nombre_ventes END) AS ventes_t2,
  concat(round(((MAX(CASE WHEN trimestre = 'T2' THEN nombre_ventes END) -
    MAX(CASE WHEN trimestre = 'T1' THEN nombre_ventes END)) /
    MAX(CASE WHEN trimestre = 'T1' THEN nombre_ventes END)) * 100,2), '%') AS taux_evolution
FROM
  VentesParTrimestre;
```

The results are displayed in a table with the following data:

	123 ventes_t1	123 ventes_t2	A7 taux_evolution
1	16 838	17 457	3.68%

The bottom status bar indicates that 1 row(s) were fetched, with a fetch size of 0.04s, on 2026-01-18 at 19:00:11.

Requête 8

Le classement des régions par rapport au prix au mètre carré des appartement de plus de 4 pièces.

The screenshot shows the DBeaver SQL editor interface. The left sidebar displays the database structure for 'SGBD-MySQL'. The main editor area contains the following SQL query:

```
/*8. Le classement des régions par rapport au prix au mètre carré des appartement de plus de 4 pièces.*/
select
  r.reg_nom as region,
  round(avg(v.valeur/ b.surface_carrez_lot), 0) as PrixMoyenmetrecarre
from
  vente v
join
  bien b on v.Identifiant_bien = b.Id_bien
join
  departement d on d.dep_code = b.Depcode
join
  region r on r.reg_code = d.dep_reg_code
where
  b.biencodetypelocal = 2
  and b.Total_pieces > 4
group by
  region
order by PrixMoyenmetrecarre desc
;
```

The results are displayed in a table with the following data:

region	PrixMoyenmetrecarre
1 Ile-de-France	8770
2 La Réunion	3642
3 Provence-Alpes-Côte d'Azur	3588
4 Corse	3105
5 Auvergne-Rhône-Alpes	2891
6 Nouvelle-Aquitaine	2465
7 Bretagne	2412
8 Pays de la Loire	2316
9 Hauts-de-France	2190
10 Occitanie	2097
11 Normandie	2016
12 Grand Est	1541
13 Centre-Val de Loire	1453
14 Bourgogne-Franche-Comté	1251
15 Martinique	573

The bottom status bar indicates that 15 rows were fetched, and the query was executed on 2026-01-20 at 23:19:51.

Requête 9

Liste des communes ayant eu au moins 50 ventes au 1er trimestre jusqu'à mars

The screenshot shows the DBeaver 25.3.0 interface with a MySQL database named 'SGBD-MySQL'. The 'baseimmobiliere' database is selected, and the 'ventes' table is visible in the left sidebar. The main editor displays a SQL query that filters for communes with at least 50 sales in the first quarter (January to March) and orders them by the number of sales in descending order.

```
/*9. Liste des communes ayant eu au moins 50 ventes au 1er trimestre jusqu'à mars ==> 48 lignes dont ajaccio 54 trié pr nom (voir vincennes 68) */  
SELECT  
  c.com_nom,  
  count(*) as N  
FROM  
  vente v  
JOIN  
  bien b ON b.Id_bien =v.Identifiant_bien  
JOIN  
  commune c ON c.com_code = b.comcode and c.com_coddep =b.Depcode  
WHERE  
  MONTH(v.date) in (1,2,3)  
  and v.valeur is not null  
GROUP BY  
  c.com_nom  
HAVING  
  count(*) >= 50  
ORDER BY  
  c.com_nom desc
```

The results are displayed in a table with two columns: 'A2 com_nom' and '123 N'. The table lists 48 communes, with Vincennes having the highest number of sales (68) and Paris 18e Arrondissement having the lowest (210).

A2 com_nom	123 N
Vincennes	68
Versailles	54
Toulouse	79
Toulon	59
Sète	62
Saint-Maur-des-Fossés	56
Rueil-Malmaison	68
Rennes	62
Puteaux	53
Paris 9e Arrondissement	106
Paris 8e Arrondissement	62
Paris 7e Arrondissement	87
Paris 6e Arrondissement	86
Paris 5e Arrondissement	79
Paris 4e Arrondissement	59
Paris 3e Arrondissement	59
Paris 2e Arrondissement	61
Paris 20e Arrondissement	130
Paris 19e Arrondissement	117
Paris 18e Arrondissement	210

Requête 10

Différence en pourcentage du prix au mètre carré entre un appartement de 2 pièces et un appartement de 3 pièces.

The screenshot shows the DBeaver SQL editor interface. The left sidebar displays the database structure for 'SGBD-MySQL' and 'baseimmobiliere'. The main editor contains a SQL query to calculate the percentage difference in price per square meter between 2-room and 3-room apartments. The query uses a subquery to calculate the average price per square meter for each room type, then uses a CASE statement to calculate the percentage difference.

```
/*10. Différence en pourcentage du prix au mètre carré entre un appartement de 2 pièces et un appartement de 3 pièces */
WITH PrixMoyenMetreCarre AS (
  SELECT
    b.Total_pieces as Totalpieces,
    round(avg(v.Valeur)/ avg(b.Surface_local),0) AS prix_metre_carre
  FROM
    vente v
  JOIN
    bien b ON b.Id_bien = v.Identifiant_bien
  WHERE
    b.biencodetypelocal =2 and b.Total_pieces in (2,3) and YEAR(v.date) = 2020 AND MONTH(v.date) IN (1, 2, 3, 4, 5, 6)
  GROUP BY
    b.Total_pieces
)
SELECT
  CONCAT(ROUND(
    (MAX(CASE WHEN Totalpieces = 2 THEN prix_metre_carre END) -
     MAX(CASE WHEN Totalpieces = 3 THEN prix_metre_carre END)) /
    MAX(CASE WHEN Totalpieces = 2 THEN prix_metre_carre END) * 100, 2 ),'%') AS DifferencePrixMoyenMetreCarre
  FROM
    PrixMoyenMetreCarre
;
```

The results pane shows a single row with the value 11.43%.

AZ DifferencePrixMoyenMetreCarre
11.43%

The status bar at the bottom indicates that 1 row(s) were fetched on 2026-01-21 at 00:15:15.

Requête 11

Les moyennes de valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69

The screenshot shows the DBeaver SQL editor with a query for the top 3 communes by average land value in departments 6, 13, 33, 59, and 69. The query uses window functions like RANK() and AVG(). The results table shows 12 rows of data with columns for department code, commune name, and average value.

```
/*11. Les moyennes de valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69.*/
WITH Moyenne_valeur_par_commune AS (
  SELECT c.com_coddep as code_departement, c.com_nom as nom_commune, round(avg(v.valeur),2) as valeur
  FROM vente v
  JOIN bien b on v.Identifiant_bien = b.Id_bien
  JOIN commune c on c.com_coddep = b.depcode and b.comcode = c.com_code
  WHERE c.com_coddep IN (06,13,33,59,69)
  GROUP BY c.com_coddep , c.com_nom
)
SELECT code_departement , nom_commune , valeur
FROM (
  SELECT code_departement , nom_commune , valeur ,
  rank() OVER (PARTITION BY code_departement ORDER BY valeur DESC) as rang
  FROM Moyenne_valeur_par_commune
) as result
WHERE rang <= 3
```

The results table shows the following data:

A2 code_departement	A2 nom_commune	123 valeur
06	Saint-Jean-Cap-Ferrat	968 750
06	Eze	655 000
06	Mouans-Sartoux	476 898,09
13	Gignac-la-Nerthe	330 000
13	Saint-Savournin	314 425
13	Cassis	313 416,88
33	Lège-Cap-Ferret	549 500,64
33	Vayres	335 000
33	Arcachon	307 435,93
59	Bersée	433 202
59	Cysoing	408 550
59	Halluin	322 250

Requête 12

Les 20 communes avec le plus de transactions pour 1000 habitants pour les communes qui dépassent les 10 000 habitants.

The screenshot shows the DBeaver 25.3.0 interface. The left sidebar displays the database structure for 'baseimmobiliere', including tables like 'bien', 'commune', 'population', and 'vente'. The main editor contains a SQL query that calculates the transaction rate per 1000 inhabitants for communes with more than 10,000 inhabitants, ranking them by this rate. The bottom pane shows the results in a table with 20 rows.

```
SELECT
  j.com_nom AS classement_communes,
  round( 1000.0 * count(j.Indentifiant_vente) / p.popu_totale , 2) AS Taux1000,
  RANK() OVER (ORDER BY COUNT(j.Indentifiant_vente) DESC) AS Top_communes
FROM
  jointure j
JOIN
  population p ON p.pop_com_code = j.com_code and p.pop_com_dep = j.com_coddep
WHERE
  p.popu_totale > 10000
GROUP BY
  j.com_nom ,p.popu_totale
)
SELECT
  classement_communes,
  Taux1000
FROM
  RankedCommunes
ORDER BY
  Taux1000 desc
limit 20
;
```

	AZ classement_communes	123 Taux1000
1	Paris 2e Arrondissement	5,84
2	Montévrain	5,72
3	Paris 1er Arrondissement	4,86
4	Paris 3e Arrondissement	4,69
5	Arcachon	4,62
6	La Baule-Escoubiac	4,58
7	Paris 4e Arrondissement	4,05
8	Roquebrune-Cap-Martin	3,99
9	Paris 8e Arrondissement	3,83
10	Sanary-sur-Mer	3,61
11	La Londe-les-Maures	3,43
12	Paris 9e Arrondissement	3,43
13	Paris 6e Arrondissement	3,38
14	Saint-Cyr-sur-Mer	3,16
15	Chantilly	3,13
16	Pornichet	3,06
17	Saint-Mandé	3,06
18	Paris 10e Arrondissement	3,04
19	Menton	2,94
20	Saint-Hilaire-de-Riez	2,87

20 row(s) fetched - 0,447s, on 2026-01-21 at 00:29:10



Merci !

Compétences	Livrables	Indicateurs de réussite de l'activité	Notes
Créer une base de données en respect des normes réglementaires et des besoins clients	Le support de présentation	☒ Je suis satisfait de mon schéma relationnel présent dans le support. ☒ Il suit correctement la norme 3NF. ☒ Il contient des données atomiques. ☒ Il est composé de l'ensemble des données nécessaires pour le projet.	
	Le dictionnaire des données	☒ Je suis satisfait de mon dictionnaire des données : il contient toutes les données nécessaires. ☒ J'ai répertorié les variables selon les informations à ma disposition. ☒ Mes données sont atomiques. ☒ Mes données sont correctement décrites. ☒ Je n'ai pas inclus dans le dictionnaire de données personnelles conformément au RGPD.	
Gérer une base de données établissant des requêtes et en respect du RGPD	La base de données présentée dans le support de présentation	J'ai vérifié que : ☒ mes tables correspondent au schéma relationnel ; ☒ les contraintes d'intégrité sont présentes ; ☒ les types de variables sont logiques et respectent le dictionnaire des données ; ☒ ma base de données respecte le RGPD ; ☐ ma base de données contient un nombre de lignes cohérent que je peux justifier ;	

		☒ les données sont intégrées dans les colonnes correspondantes.	
Effectuer des requêtes SQL pour répondre à une problématique métier en respect du RGPD	La présentation avec les requêtes et les résultats	☒ Il existe une requête et une réponse pour chaque demande présente. ☒ Les 3 requêtes fonctionnent dans la base de données et renvoient des résultats justes. ☒ Je peux justifier l'utilisation des alias. ☒ Je suis satisfait de mon code SQL. ☒ J'ai utilisé des sous-requêtes ou des tables temporaires et je peux le justifier.	